PROJECT AND TEAM INFORMATION

Project Title

Plagiarism Detection System

Student / Team Information

Team Name: *Team #*	***Average_Coders***
Team member 1 (Team Lead) *(Last Name, name: student ID: email, picture):*	*Sharma, Anushka : 230221418* *s10.anushka@gmail.com* 
Team member 2 *(Last Name, name: student ID: email, picture):*	*Mishra, Keshav:230213613* *keshavmishra1404@gmail.com* 

Team member 3 *(Last Name, name: student ID: email, picture):*	*Gupta, Aditya : 23021115* *adityaguptaaitech79@gmail.com* 

PROPOSAL DESCRIPTION

Motivation

Programming projects are a key component of teaching and assessing student's understanding of coding principles in academic settings. However, plagiarism in source code submissions has become a regular issue due to the simplicity with which programs can be copied from classmates or other sources. This makes it difficult for teachers to accurately evaluate pupils' individual efforts and learning achievements.[1]
Manually reviewing code submissions takes time and is often impracticable, especially when there are a high number of students. Teachers may overlook instances of copied or slightly modified code, which can affect academic integrity. Simultaneously, basic programming issues tend to produce similar solutions, making it difficult to separate actual labor from cloned code.

The goal of this project is to provide a simple and fast system that helps teachers find unusually high similarity in programming assignments. By automatically analyzing code submissions and determining similarity percentages, the technology decreases the amount of manual effort necessary for plagiarism detection while producing consistent and objective results. [4]

This project is significant because it promotes fair grading, encourages honest learning techniques, and allows students to focus on comprehending topics rather than reproducing solutions. The system is intended to be simple to use, deployable locally, and appropriate for college-level contexts, making it a useful tool for academic purposes.[1]

State of the Art / Current solution

Currently, plagiarism in programming assignments is tackled both manually and automatically. Many institutions require teachers to manually analyze student's code submissions to find similar patterns, which takes time and becomes impractical as the number of assignments increases. Manual verification also relies primarily on the professor's experience and he may overlook minor instances of plagiarism.

To solve these challenges, numerous automatic plagiarism detection technologies exist, including code similarity checkers and online plagiarism detection platforms. To find program similarities, these tools often use techniques such as token comparison, string matching, and structural analysis. Some advanced systems employ Abstract Syntax Trees (AST) and similarity metrics to detect copied logic even when variable names or formatting change. However, many existing technologies are either cost effective, need internet access, or function as black-box solutions in which the comparison process is not accessible to teachers or students. This limits their access and flexibility in college settings.[3]

As a result, there is a need for easy, transparent, and locally deployable plagiarism detection solutions that can help educators discover suspicious code similarity while being flexible for academic purposes.

Project Goals and Milestones

The main purpose of the Plagiarism Detection System is to help professor's find similarities in source code/assignments submissions in an effective and fair manner. The application's goal is to automatically evaluate programming assignments, assess similarity percentages, and indicate suspected examples of plagiarism, all while allowing natural resemblance in simple issues.[4]

Another major goal is to create an application that is simple to use, deploy locally, and appropriate for academic settings. The project also intends to provide a clear and transparent plagiarism analysis method, allowing instructors to analyze results, check the output(tokens), review similarities and provide feedback to students. Furthermore, the application is meant to be modular, which allows for future additions such as improved similarity algorithms or database integration.[3]

PROJECT MILESTONES:

Requirement Analysis: Investigated the topic of code plagiarism and analyzed existing ways to determine appropriate similarity detecting algorithms[4]

System Design: Developed the general architecture, which included frontend, backend, and data storage components.[1]

Backend Development: We will Python and Flask to implement essential functions such as file uploads and similarity computing.[2]

Plagiarism Detection Logic: Creation and testing of the main logic for comparing code submissions and assigning similarity scores (in progress).[2]

Frontend Development: We plan on using HTML and CSS user interfaces for file uploading, displaying results, and receiving comments.[5]

Data Storage Integration: JSON-based storage to manage submissions and plagiarism results.[8]

Testing and deployment: We will use Multiple sample files to test the application, which will then be installed locally for demonstration and evaluation.

Project Approach

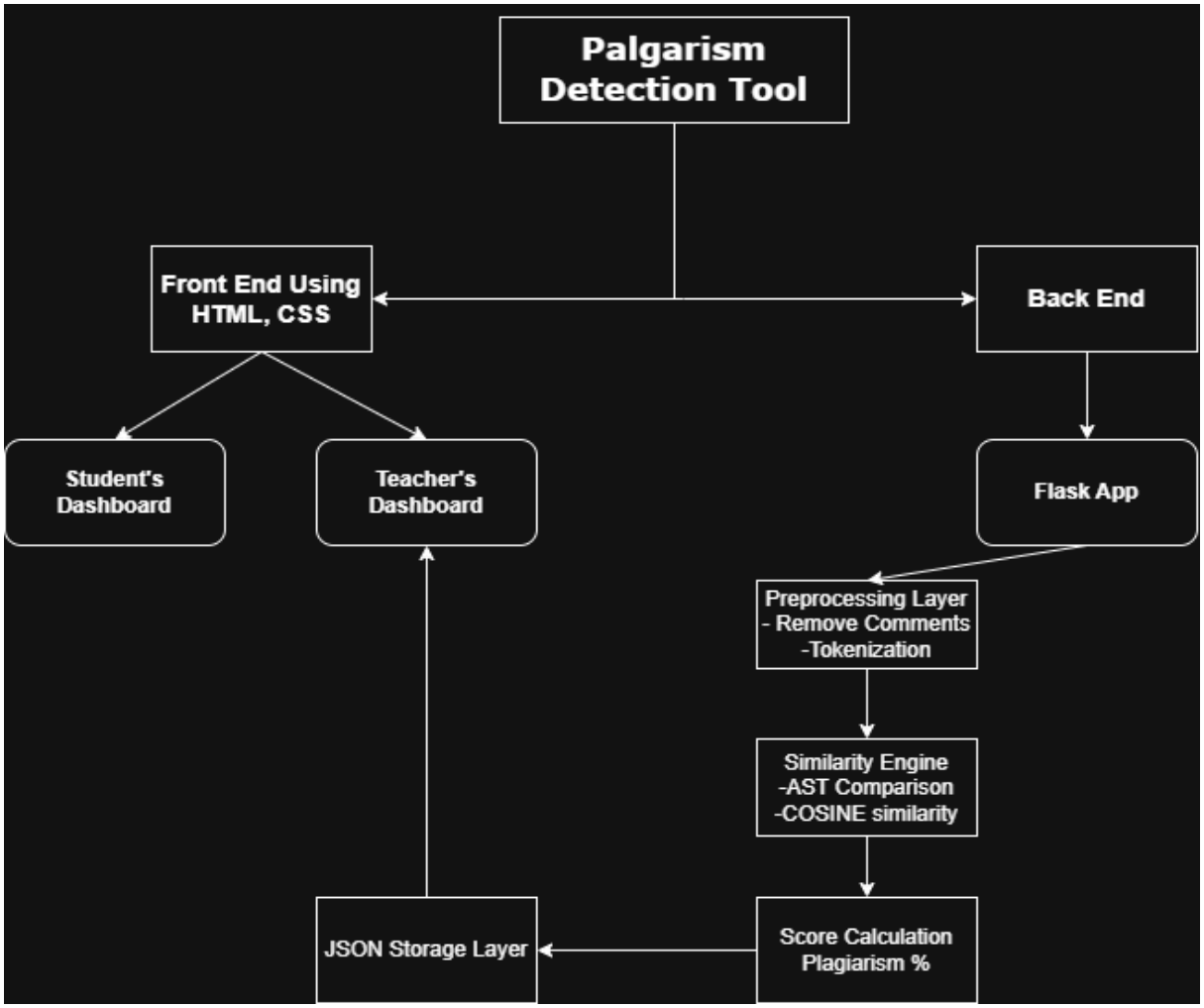
The project uses a structured and practical approach for designing and implementing a Plagiarism Detection System for source code/assignments submissions. The solution is designed as a web-based application that enables users to easily interact with the system.

The backend is built with Python and the Flask framework, which was chosen for its ease of use and flexibility in handling web requests. Flask is used to handle file uploads, requests, and similarity analysis. The program interacts over normal HTTP GET and POST protocols, making it simple to learn and deploy locally.[5]

The plagiarism detection technique starts with gathering source code files submitted by students. These files are saved locally and then analyzed to compare them to previous submissions. The primary approach entails assessing code similarity by tokenizing the content and then reading and comparing these tokens in order to identify exceptionally high levels of similarity. A baseline similarity notion is being developed in order to avoid mistakenly detecting simple and often solved systems.[6]

The system's frontend is built with HTML and CSS, making it easy to upload code and view results. A JSON-based storage technique is used to keep submission details, plagiarism scores, and feedback, reducing the need for sophisticated database systems.The system is modular in design, with the user interface, processing logic, and data storage all separated. This technique enables for simple maintenance and potential enhancements, such as incorporating advanced similarity algorithms or using a database for larger datasets.[7]

System Architecture (High Level Diagram)



Project Outcome / Deliverables

This project produced a fully functional Plagiarism Detection System capable of comparing source code submissions and identifying program similarities. The system allows users to upload code files, which are then automatically analyzed to produce a similarity percentage using code comparison.

The project produces a web-based application built with Python and Flask, as well as a simple user interface designed with HTML and CSS. The system successfully saves submission information, plagiarism results, and feedback utilizing a lightweight JSON-based storage approach. Instructors can view similarity results and provide feedback to students using the system. The project also includes tested plagiarism detection logic capable of detecting exceptionally high resemblance between programming/assignments submissions while allowing for natural similarity for basic tasks. The application is locally deployable and appropriate for academic settings.[5]

In addition to the working software, the project contains documentation outlining the system's concept, methodology, and use. These deliverables, taken combined, provide a useful tool for educators to help them preserve academic integrity and evaluate student programming work more quickly.

Assumptions

The system assumes that all uploaded files are genuine source code (in python format) /assignments and readable in text format. It is believed that students submit their programs/assignments separately, and that all contributions address the same or similar programming problems. The plagiarism detection process considers that greater similarity than a normal baseline may suggest plagiarism. The system also assumes local deployment with a limited number of concurrent users and data storage in JSON files, making it appropriate for small to medium-sized academic applications.[4]

References

[1]. Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D., Compilers: Principles, Techniques, and Tools, Pearson Education.
[2]. Flask Documentation – https://flask.palletsprojects.com
[3]. Python Official Documentation – https://docs.python.org
[4]. Chudnovsky, M., "Plagiarism Detection in Programming Assignments," ACM Computing Surveys.
[5]. Manning, C. D., Raghavan, P., & Schütze, H., Introduction to Information Retrieval, Cambridge University Press.
[6]. Python Scikit-learn Documentation – Cosine Similarity
https://scikit-learn.org/stable/modules/metrics.html#cosine-similarity
[7]. Levenshtein, V. I., "Binary Codes Capable of Correcting Deletions, Insertions, and Reversals," Soviet Physics Doklady.
[8]. GitHub Open Source Examples and Tutorials related to Flask and Python programming.